Raw-Python vs. LangChain Agent: Comparison

Goal

Rebuild the raw-Python ReAct agent (calculator, weather, and game-lookup tools) using LangChain with Groq as the LLM provider, then compare loop control, tool handling, memory, and error handling between the two implementations, using an actual execution trace as evidence rather than a general framework description.

Setup

Both agents use the same three tools and the same overall reasoning shape (select tool, execute, feed the result back, produce a final answer). The LangChain version adds LCEL chains, RunnableWithMessageHistory for multi-turn memory, a submit_recommendation tool bound to a Pydantic schema for structured output, and a deliberately unreliable flaky_game_price_lookup tool to test failure recovery.

Implementation Comparison

Component	Raw Python	LangChain
Loop control	Manual while True loop; checked model output for tool calls, dispatched, appended results	AgentExecutor manages the act/observe/respond loop internally
Tool registration	if tool_name == ... dispatch logic with hand-written schemas	@tool decorator infers schema from function signature and docstring
Conversation memory	Manually appended assistant/tool messages to a list	RunnableWithMessageHistory + session-keyed history store (now deprecated in favor of LangGraph persistence)
Behavior instructions	system_prompt string passed directly into the message list	ChatPromptTemplate with MessagesPlaceholder("agent_scratchpad") / ("chat_history")
Model/version logging	response.model printed after every call	Not exposed by default; needs custom callbacks or tracing
Validation logic (regex, strip_emojis)	Plain Python, unchanged	Same Python, wrapped as a LangChain tool

Annotated Reasoning Trace

Captured from a live AgentExecutor run (verbose=True) on the request: "What's the weather in Lahore, and how long does it take to finish Hades?"

Reason: Model identifies the request needs both weather data and game data, two separate tool calls.

Action 1: get_weather({"city": "Lahore"})

Observation 1: "Lahore: Sunny, 38°C"

Action 2: lookup_game({"title": "Hades"})

Observation 2: Hades (2020): Roguelike by Supergiant Games, PC/Switch/PS5/Xbox Series X|S, avg completion 21 hours.

Final answer: Model merges both observations into one natural-language response, closing the loop without further tool calls.

This matches the reason → act → observe pattern the raw-Python agent implemented manually. The difference is where that loop lives: AgentExecutor owns it internally, so verbose=True shows the tool calls and observations but not the formatted prompt or the model string returned per call — that level of detail needed explicit logging in raw Python and would need callbacks/tracing here.

Issues Observed

- Redundant tool calls: in the three-turn memory test, turn 2 ("What about Portal 2?") called lookup_game("Portal 2") twice inside a single AgentExecutor run before producing the final answer, no failure triggered the retry, it just re-invoked the same tool with the same input.
- Instruction drift on structured output: the system prompt explicitly restricted submit_recommendation to cases where the user "asks you to choose, recommend, or pick one game." In the final integration test, turn 2 ("Now compare that to Portal 2") is a comparison request, not a recommendation ask, yet the agent invoked submit_recommendation anyway. Because this call is structured, it was easy to catch; the same drift in free-text output would have gone unnoticed.
- RunnableWithMessageHistory raised a LangChainDeprecationWarning at run time, pointing to LangGraph persistence as the replacement, so the memory pattern used here is already legacy in the current LangChain version.
- handle_tool_error was not supported by the installed @tool decorator version, so the flaky-tool failure had to be caught and converted to a message manually inside the function body instead of relying on the documented parameter.

Conclusion

LangChain removes the boilerplate for loop control, tool schema definition, and history wiring, and that reduction is real and consistent across every section of the notebook. What it costs is direct visibility: the raw-Python agent's manual loop made state, retries, and rule violations obvious at the point they happened, while the LangChain agent needed the structured-output tool call to surface a rule violation that would otherwise have been invisible in the final text. For a small, fixed tool set, the raw-Python version is easier to audit; for anything that needs to scale past a handful of tools or add multi-turn memory quickly, LangChain's boilerplate reduction outweighs that visibility loss, provided the debugging gap is closed with logging or tracing rather than assumed away.